AWS Trading Platform

Scope of Work

Exchange architecture, AI Guardian and dynamic hot wallet controls

Prepared by Dzoan Nguyen Tran | DevOps and Cloud Engineer | 12 November 2026

This proposed scope translates the DevOps challenge into a reviewable design and staged proof of concept. It defines the platform, AWS operations, trading safety and an isolated wallet simulator. It is not a production custody authorization or a commitment to a live launch.

Intended readers

DevOps challenge reviewers, platform architects, security engineers, trading operations and custody stakeholders. The scope, capacity and business policies require validation before delivery.

Project summary and demo

This design proposes a resilient spot-trading platform on AWS, using EKS across Availability Zones, Aurora PostgreSQL, an AI Guardian and simulated compartmentalized wallets. Prometheus and Grafana support operations; cross-Region backups support disaster recovery. Service alternatives, scaling and cost tradeoffs are documented above.

Demo link: [Insert the deployed demo or public repository URL here]. A working demo has not yet been provided or verified.

The challenge requests an architecture diagram, service choices with alternatives and a scaling plan. Implementation, performance and recovery outcomes must be reported only after testing.

1 Challenge requirements

Architecture a highly available trading system with similar features to the Binance trading platform https://www.binance.com. This system will need to be resilient to failures, scalable, and cost-effective. You should design the architecture, choose the appropriate technologies, and explain your approach for maintaining high availability and scalability.

Due to time constraint, you will not be able to cover every feature on the reference platform, so choose some features that will help you demonstrate your mastery of cloud infrastructure.

Deliverables

Your submission should include:

- An overview diagram of the services used and what role they play in the system.
- Elaboration on why each cloud service is used and what are the alternatives considered.
- Plans for scaling when the product grows beyond your current setup.

You are limited to the following constraints for your plan.

Requirement	What the submission must demonstrate
Cloud provider	Use Amazon Web Services (AWS).
Trading platform	Design a system with selected features similar to a cryptocurrency trading platform.
Throughput	Support 500 requests per second.
Response time	Target p99 response time below 100 ms. Define the measurement conditions and explain how to test them.
Failure resilience	Explain how the system responds to component and infrastructure failures.
High availability	Explain how services remain available or recover after a failure.
Scalability	Explain how the platform can handle growth beyond the initial workload.
Cost-effectiveness	Select services and capacity that balance cost, availability and performance.

2 Features Assumptions for our Binance like experience

The Guardian and hot-wallet controls are our proposed additions; they are not explicitly required by the challenge. The wallet feature is scoped as a simulation or testnet demonstration, with no real customer funds or private keys.

Feature	Purpose
Spot trading	Submit and cancel selected limit orders, match orders and view their status.
Balances and ledger	Track available and reserved balances and record trade-related financial movements.
Market data	Show prices, order-book updates and recent trades.
AI Trading Guardian	Detect unusual activity, create risk alerts and support human review.
Safety guardrails	Apply order limits, balance checks, idempotency and controls to prevent duplicate or unauthorized actions.
Dynamic hot-wallet management	Simulate separate wallet compartments per asset and network, with liquidity targets, caps and controlled approvals.
Monitoring	Use Prometheus, Grafana and Alertmanager to monitor performance, failures, trading health and risk.

2 Scope and boundaries

In scope for architecture and simulated pilot

- Spot limit orders, cancellation, matching for selected markets, order history, balance reservation, basic market data and simulated settlement.
- AWS EKS with multi AZ worker nodes, ingress, Aurora PostgreSQL, cache, queues, GitLab CI/CD, Helm or GitOps and CloudFormation.
- Metrics with Prometheus, Grafana and Alertmanager; API, matching, wallet exposure, queue lag, security and ledger monitoring.
- Guardian rules, synthetic behavioral signals, review queue and simulated pause controls; optional model evaluated in shadow mode.
- Simulated or testnet deposits and withdrawals, isolated asset/network wallet pools, liquidity calculation, mocked signer and human approved transfer workflow.
- Cross Region backup copies, infrastructure recreation, recovery runbook, data reconciliation and game day tests.

- Explicitly out of scope for pilot
- Live customer funds, real private keys, production custody, direct connection to production exchanges or automatic on chain treasury transfers.

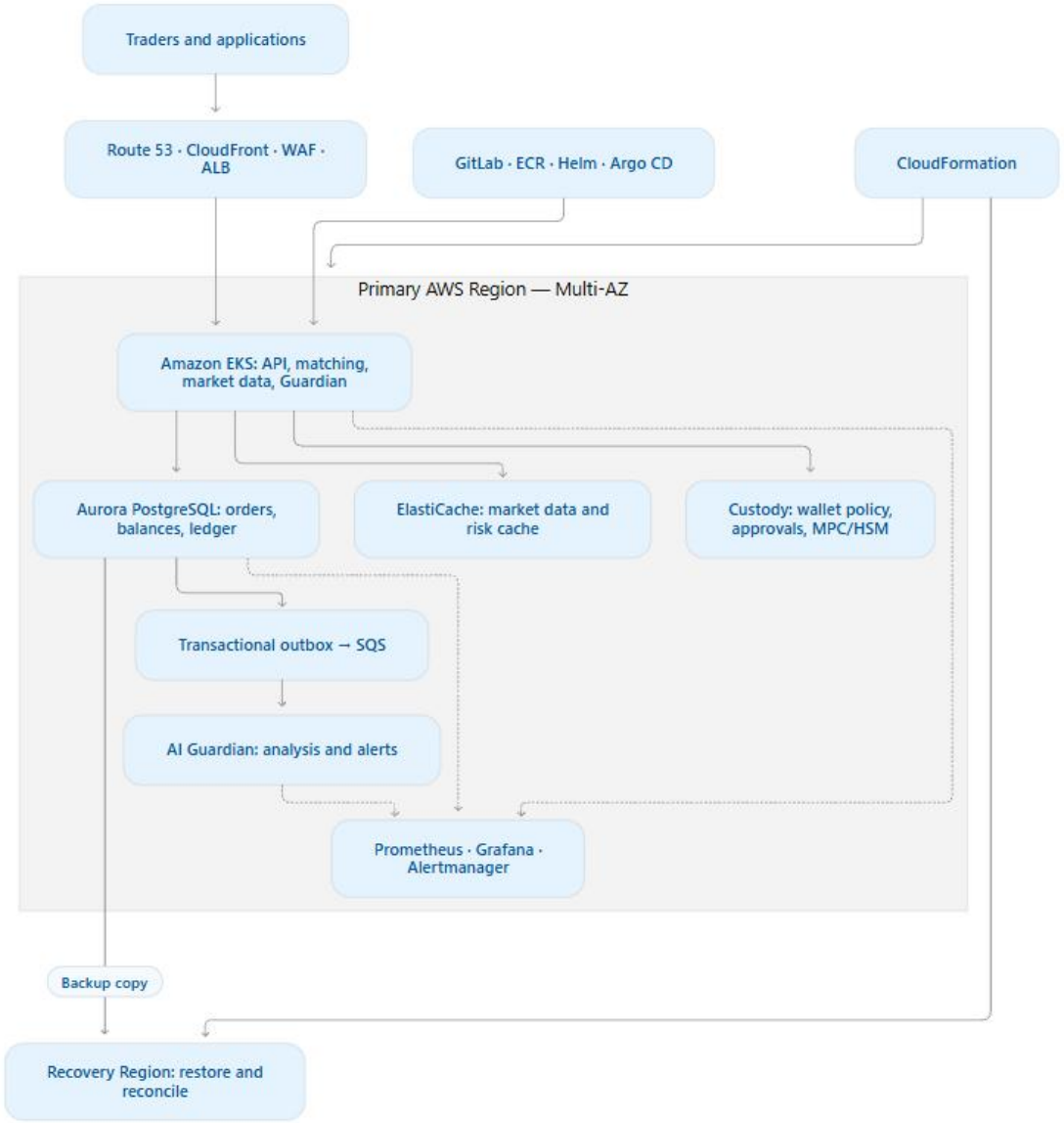
- Margin, leverage, derivatives, fiat rails, KYC/AML production operations, live custody integrations, token listing, complex market surveillance and customer support. Guaranteed fraud prevention, zero loss, global sub 100 ms latency, regulatory compliance certification or a production ready exchange.
- Production deployments, 24/7 support, independent security audit and live custody launch require a separately approved scope.

3 Proposed solution on AWS

Trading and platform requirements

ID	Requirement and output
TR-01	Authenticate users and authorize market, account and order access.
TR-02	Validate order size and price increments; enforce configured limits and reserve available balance atomically.
TR-03	Durably record the order and unique client id; return ACCEPTED only after commit, not after execution.
TR-04	Process accepted commands in defined market order, prevent dual ownership with fencing, and support cancel and partial fill states.
TR-05	Record trades and balanced ledger entries transactionally; replay committed state after failures and reconcile.
TR-06	Publish a reliable outbox event for downstream read models and Guardian; consumers handle duplicate events.
TR-07	Provide market data and order history with clearly identified freshness; never treat the cache as the ledger.
TR-08	Record audit evidence: actor, action, policy version, correlation id and decision. Protect sensitive data.

Architecture overview : apps Worklow



Detailed AWS architecture

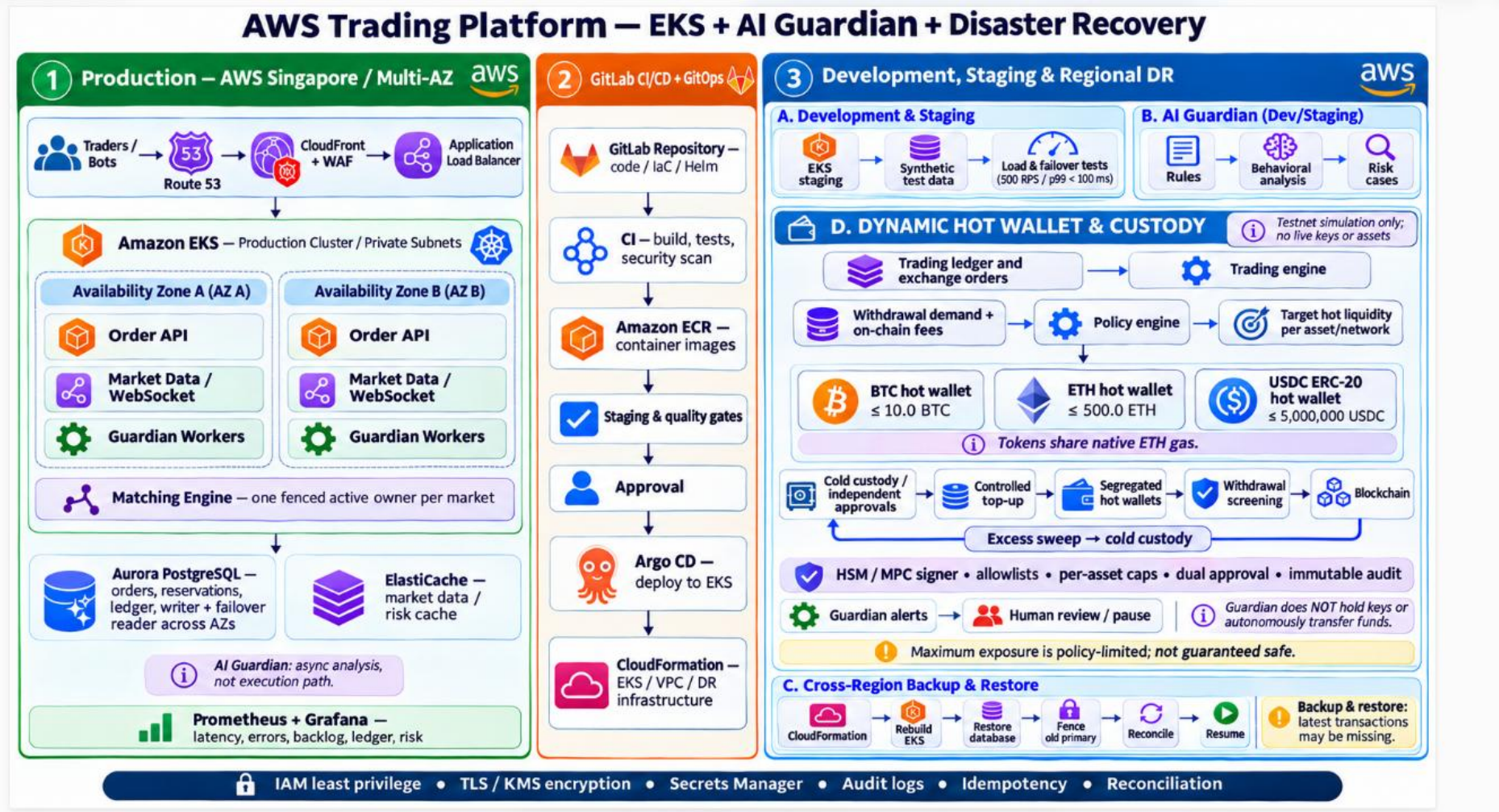


Diagram notes: wallet caps are illustrative placeholders, not recommended limits. The custody flow is simulated; fence the former primary before enabling recovery-region writes. 500 RPS and p99 are targets, not measured results.

AWS services and alternatives

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
DNS	Route 53	Managed DNS / TTL affects failover	Route 53 + Global Accelerator	Static IPs, faster regional routing / extra cost
Content delivery	CloudFront	Edge caching / configuration and transfer cost	Direct S3 or ALB	Simpler / no edge caching
Web protection	WAF on CloudFront	Filters at the edge / rules need tuning	WAF on ALB	Regional protection / traffic reaches AWS Region first
API entry	Application Load Balancer	HTTP and WebSocket routing / API controls need design	API Gateway	Managed API controls / extra cost and latency to test
Customer identity	Cognito	Managed authentication / integration constraints	External identity provider	Flexible features / vendor and integration costs
Containers	EKS	Kubernetes ecosystem / cluster operations	ECS	Simpler orchestration / less Kubernetes portability; performance depends on workload.
Compute	EKS managed EC2 nodes	Node control / capacity and patching	EKS on Fargate	Less node management / less host control
Database	Aurora PostgreSQL	Transactions and failover / cost and performance testing	RDS PostgreSQL Multi-AZ	Managed PostgreSQL / different scaling and failover

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
Cache	ElastiCache	Fast reads / cache consistency	DynamoDB	Managed key-value store / different access patterns
Events	SQS	Managed queues / limited replay	MSK	Streaming and replay / more complexity
Container registry	ECR	AWS-integrated / storage and replication costs	GitLab Registry	Integrated with GitLab / GitLab dependency
Monitoring	Prometheus + Grafana on EKS	Full control / team manages the stack	AWS Managed Prometheus + Grafana	Less administration / service charges
Infrastructure as code	CloudFormation	AWS-native deployment / template maintenance	AWS CDK	Reusable code / extra tooling; uses CloudFormation
AZ recovery	Multi-AZ EKS + Aurora	AZ resilience / higher capacity cost	Single AZ	Lower cost / AZ failure causes interruption
Regional DR	Cross-Region backup and restore	Lower cost / slower recovery and possible data loss	Aurora Global Database + pilot light	Faster recovery / higher cost; data loss still possible
Wallet custody	Vetted custody provider or MPC/HSM design	Controlled signing / provider evaluation	Custom CloudHSM-based custody	More control / greater security and operational responsibility

4 Proposed optional safety features

Custody risk and human oversight

The proposed controls address custody risk. Binance reported a major wallet breach in October 2026; the exact attack path and effectiveness of these proposed controls require independent investigation. Source: https://www.reuters.com/legal/government/gone-20-minutes-how-biggest-heist-crypto-history-unfolded-2026-10-30/

AI Guardian and human control

Fast deterministic rules act before order acceptance. The Guardian consumes committed events asynchronously, identifies suspicious patterns and creates explainable risk cases. It may propose or apply preapproved account level restrictions through a separate policy service; it cannot sign or transfer assets.

Signal	Illustrative response
Unusual order velocity	Compare with account profile, authorized bot limits and market conditions; alert or hold according to policy.
High value plus new device	Combine signals and request review or step up authentication where appropriate.
Repeated cancellations or abnormal patterns	Create a surveillance case; legitimate market making is not automatically treated as fraud.
Guardian outage or lag	Alert operators; use approved deterministic fallback and documented fail open or fail closed rules.

Pilot acceptance: test synthetic normal and suspicious scenarios, document false positives and delayed detection, review every high impact action, and log policy/model version. An ML model is optional and starts in shadow mode; no effectiveness claim is made without evaluation.

Dynamic compartmentalized hot wallet design

Objective: keep only the liquidity needed for expected near term withdrawals plus approved buffer in each hot wallet, subject to asset/network exposure caps. The system responds to withdrawal demand, on chain congestion and treasury policy. Trading volume may inform a forecast, but it is not an obligation to move funds on chain.

Asset and network compartments

Compartment	Policy and operational note
BTC on Bitcoin	Dedicated hot wallet and independent exposure cap; account for UTXO management, confirmations and transaction fees.
ETH on Ethereum	Dedicated hot wallet and cap; reserve native ETH for network fees.
USDC on Ethereum	Separate token accounting and exposure limit; ERC 20 transactions also consume ETH gas. Distinct addresses alone do not ensure key isolation.
Other assets / networks	Onboard only after asset, network, signer, custody, risk and reconciliation reviews. Same symbol on different networks requires separate treatment.

Dynamic hot-wallet controls

- **Isolate wallets:** Use separate signing keys, permissions and limits per asset and network. Segmentation reduces exposure but cannot eliminate losses.
- **Size dynamically:** Set each target balance from forecast withdrawals, fees and a buffer, within approved minimum and maximum limits. Include pending transfers.
- **Manage liquidity:** Request a top-up when funds are low; sweep sustained excess to cold custody. Apply cooldowns and transfer limits.
- **Control transfers:** Require independent approval, destination allowlists and signer-level policy checks. Pause on suspicious activity, stale data or reconciliation errors.
- **Protect keys:** Use a vetted custody provider or chain-compatible HSM/MPC signer. Never expose keys to AI, logs or CI.
- **Reconcile:** Compare blockchain balances, pending transfers, custody records and the internal ledger. Investigate differences before resuming withdrawals.
- **Pilot:** Simulate recommendations and approvals with **no real funds or keys**.

Illustrative example: A wallet holds 0.40 BTC, with 0.10 BTC reliably in transit and counted under policy. For a target of 0.85 BTC, the proposed top-up is 0.35 BTC; otherwise, hold and investigate.

5 Availability performance monitoring and disaster recovery

Requirement	Approach and evidence
Availability	EKS replicas and nodes across AZs; scheduling and disruption controls; spare capacity for AZ loss; Aurora reader and matching standby. Test node and AZ failure.
Performance	500 requests/sec mixed API workload and p99 <100 ms measured end to end within an agreed geography. Include fast guardrail checks; test wallet and WebSocket paths separately. Measure latency, errors and saturation.
Operations	Prometheus and Grafana dashboards and Alertmanager alerts: p99, errors, order backlog, queue age, ledger discrepancies, hot balances, caps, gas, reconciliation and Guardian lag. Monitoring has its own availability plan.
Regional DR	Cross Region backup and restore; redeploy EKS and apps via CloudFormation and GitOps; fence old primary; restore, reconcile and approve before restart. Snapshot RPO can be nonzero; recent records may be unavailable.
Security	IAM least privilege, network isolation, encryption, scoped identities, secrets management, image and dependency scanning, audit logging, tested incident response.

6 Scaling plan

- API and WebSocket: scale pods and EC2 nodes independently using request latency, CPU, connection counts and resource headroom.
- Matching and events: assign separate markets to workers, retain one fenced active owner per market, and scale Guardian consumers using queue age. A hot market needs its own capacity tests.
- Data: cache suitable reads; optimize transactions and scale the Aurora writer vertically before considering partitioning. Readers can lag and must not be assumed to provide immediate balance consistency.
- Availability: maintain or provision capacity in healthy AZs. Test both the 500 RPS baseline and the required degraded-mode behavior after AZ failure.

7 Starting size and indicative cost

Component	Recommended starting size	Why / how to scale	CAPEX	Estimated OPEX / month
AWS Region	Singapore, 2 AZs	Regional placement and resilience	$0	No separate Region fee; resources charged below
EKS cluster	1 production cluster	Kubernetes control plane; scale workloads on nodes	$0	~$73
EKS worker nodes	4 × m7i.xlarge, 2 per AZ	Add nodes or use larger instances after testing	$0	~$736
Order API	2 pods across AZs	Add pods as traffic grows	$0	Included in worker nodes
Market data / WebSocket	2 pods across AZs	Scale by connection and message volume	$0	Included in worker nodes
Matching engine	1 active + 1 standby per market	Scale different markets across workers	$0	Included in worker nodes
AI Guardian	2 small worker pods	Scale from queue backlog	$0	Included in worker nodes; external AI services extra

Component	Recommended starting size	Why / how to scale	CAPEX	Estimated OPEX / month
Aurora PostgreSQL	1 writer + 1 reader, db.r7g.large	Increase writer size when needed	$0	~$486 for instances; storage, I/O and backups extra
ElastiCache	1 primary + 1 replica, example cache.t4g.small Valkey	Resize after measuring cache usage	$0	~$56 for nodes; other charges extra
SQS	Queues for initial event flows	Scale consumers as backlog grows	$0	Usage-based; estimate once message volume is known
Monitoring	Prometheus, Grafana and Alertmanager on EKS	Add capacity as metrics and retention grow	$0	Compute included in nodes; storage and additional capacity extra
Regional DR	Cross-Region backups and recovery templates	Rebuild the platform after a Regional disaster	$0	Backup storage and copy charges; recovery resources extra when activated
Edge and networking	Route 53, CloudFront, WAF, ALB, NAT and data transfer	Scale with requests, traffic and connections	$0	Usage-based; estimate separately

Budget basis: approximately $1,351/month for EKS standard control plane, four On-Demand m7i.xlarge nodes, two Aurora db.r7g.large instances and two cache.t4g.small Valkey nodes, at 730 hours in Singapore. This is a partial estimate, not total cost. Storage, I/O, networking, edge, logs, backup copies, staging, support, taxes and custody are additional; confirm current prices in AWS Pricing Calculator. $0 CAPEX means no cloud hardware purchase, not zero engineering cost. Sizing and performance remain unverified.

8 Delivery phases and outputs

Estimates are illustrative and assume a small team, access to AWS accounts and timely business input. They are not contractual commitments; phases can overlap.

Phase	Indicative duration	Output
0. Discovery	1–2 weeks	Confirmed requirements, traffic assumptions, risk policy, architecture and test plan.
1. Trading foundation	2–3 weeks	Simulated orders and ledger, EKS deployment, CI/CD and basic dashboards.
2. Guardian and wallet simulator	2–3 weeks	Rule-based Guardian, simulated wallet compartments, sizing, approvals and reconciliation.
3. Performance and cost evaluation	1–2 weeks	Load-test 500 RPS and p99 <100 ms; identify bottlenecks, measure AWS costs and recommend sizing changes.
4. Scaling	1–2 weeks	Scale EKS pods and nodes, database and event workers; test higher traffic and document scaling limits and costs.
5. Resilience and validation	1–2 weeks	AZ failure and Regional recovery tests, security checks, evidence and operational runbooks.
6. Production readiness	Separate scope	Independent custody and security review, production controls, operating model and launch decision.

9 Acceptance criteria and test evidence

- **Trading:** Orders process correctly; ledger balances and retries create no duplicates.
- **Performance:** Achieve **500 RPS and p99 <100 ms** under agreed test conditions; report bottlenecks.
- **Availability:** AZ and matching-worker failures cause no duplicate trades; record recovery time.
- **Guardian:** Risk cases produce traceable alerts; AI cannot access keys or sign transactions.
- **Wallets:** Simulated targets respect limits, pending transfers, approvals and gas requirements.
- **Safety:** Test failure scenarios; pause or escalate according to policy.
- **DR:** Restore cross-Region backups; measure recovery time and data loss, reconcile and obtain approval before resuming.
- **Handoff:** Deliver diagrams, code, test results, dashboards, runbooks and outstanding gaps.

10 Assumptions and risks

- 500 RPS and p99 <100 ms are targets under an agreed request mix, client geography and error threshold; they have not been benchmarked.
- Multi-AZ supports recovery from an AZ outage; cross-Region snapshots can omit recent accepted transactions and require reconciliation.
- The pilot uses simulated funds and signer behavior; separate keys and permissions reduce, but cannot eliminate, shared-control risk.
- The initial cost excludes variable and supporting charges; validate sizing and costs with workload and failover tests.